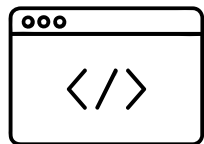
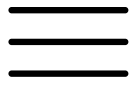


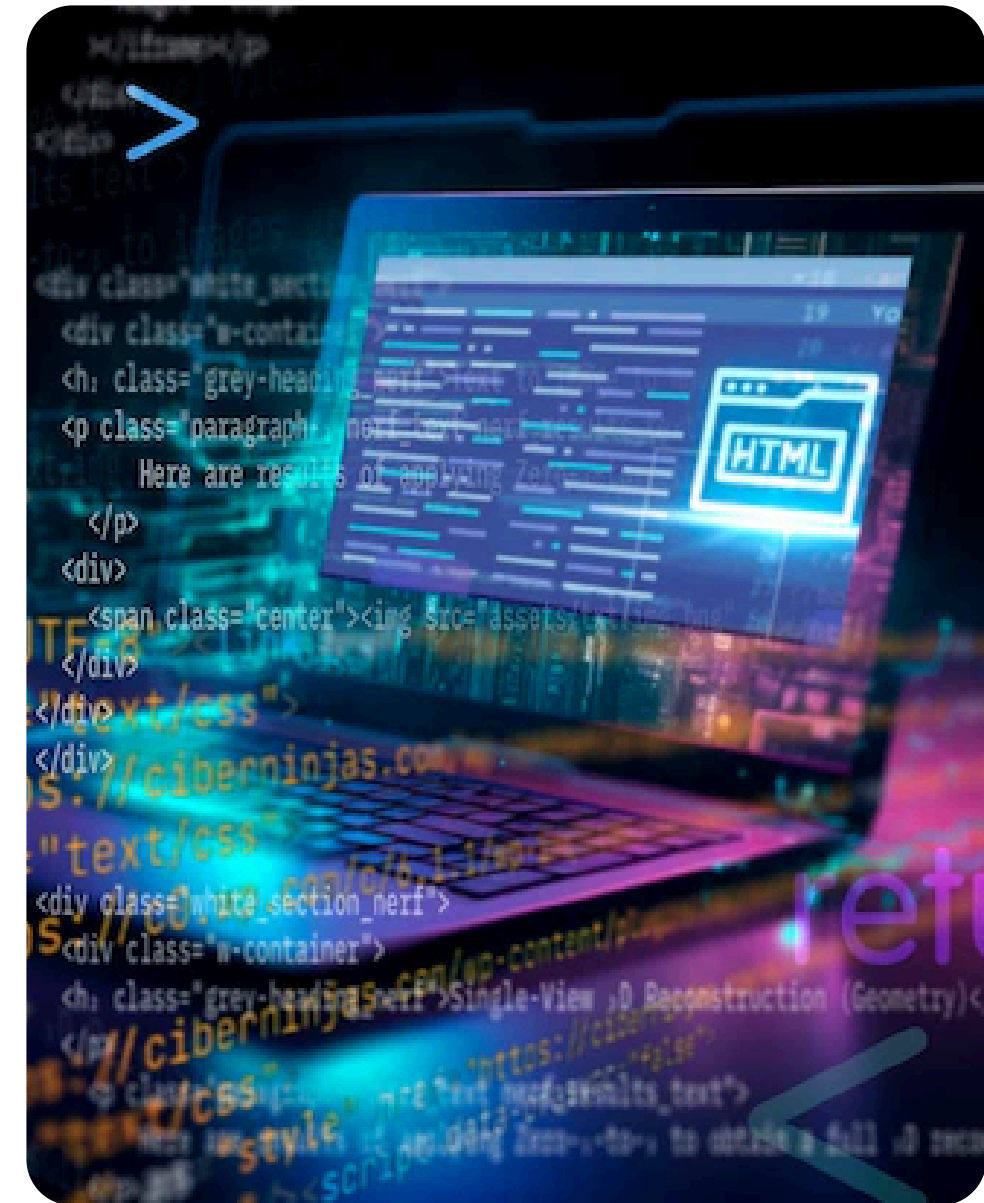
Archivos

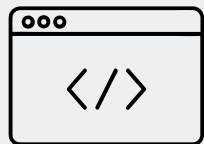
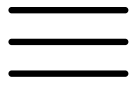
Programación Avanzada



Agenda:

1. **structs: Sobrecarga de operadores**
2. Archivos de texto
 - a. Lectura de archivos
 - b. Escritura de archivos
3. Archivos binarios
 - a. Lectura de archivos
 - b. Escritura de archivos
 - c. Funciones de acceso aleatorio





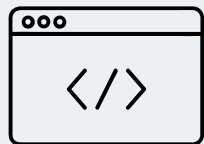
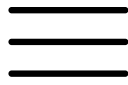
Operadores

Al crear estructuras de datos nuevas no existe una definición para los operadores básicos, por lo que es necesario definir un proceso para cada uno de ellos.

```
struct Contact {  
    string name;  
    string number;  
};
```

```
int main() {  
    Contact c;  
    cout << c << endl;  
    return 0;  
}
```

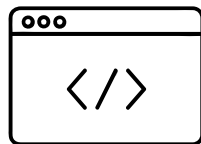
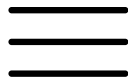
```
error: no match for 'operator<<' (operand types are 'std::ostream' {aka 'std::basic_os...  
note: candidate: 'std::basic_ostream<_CharT, _Traits>::__ostream_type& std::basic_ostr...
```



Operadores

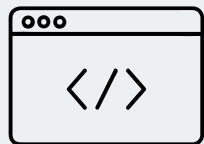
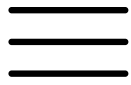
Se define el comportamiento del operador como una función dentro de la estructura con el retorno esperado después de aplicar el operador en esta.

```
struct Contact {  
    string name;  
    string number;  
    // Constructor  
    Contact() {  
        this->name = "NN";  
        this->number = "###";  
    }  
    // Operadores  
    friend ostream& operator << (ostream& os, Contact c) {  
        os << "nombre: " << c.name << endl;  
        os << "#: " << c.number << endl;  
        return os;  
    }  
};
```



Archivos de texto

Programación Avanzada



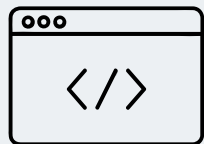
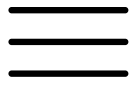
Escritura

1. Librería fstream
2. declaración de la variable
3. validar archivo
4. operador de escritura (<<)
5. cerrar archivo

```
#include <iostream>
#include <fstream> 1

using namespace std;

int main() {
    2 fstream archivo("archivo.txt", ios::out);
    int valor = 100;
    3 if(archivo) {
        4     archivo << valor;
        5     archivo.close();
    }
    return 0;
}
```



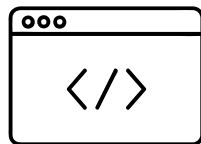
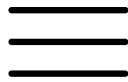
Lectura

- 1.Librería fstream
- 2.declaración de la variable
- 3.validar archivo
- 4.operador de lectura (<<)
- 5.cerrar archivo

```
#include <iostream>
#include <fstream> 1

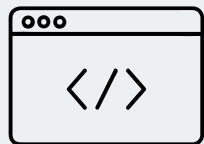
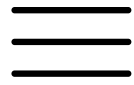
using namespace std;

int main() {
    2 fstream archivo("archivo.txt", ios::in);
    int valor;
    3 if(archivo) {
    4     archivo >> valor;
        cout << valor << endl;
    5     archivo.close();
    }
    return 0;
}
```



Archivos binarios

Programación Avanzada

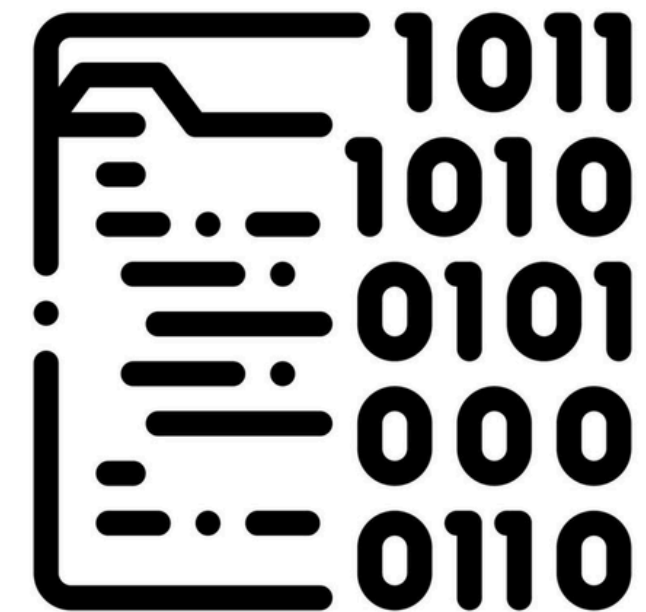


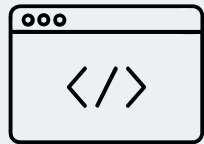
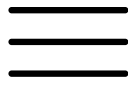
Archivos binarios

Formato de archivo basado en bits para almacenar de manera eficiente valores, almacenándolo de la manera exacta en la que se encuentran en memoria.

Extensiones:

- .bin
- .dat
- .jpg
- ...





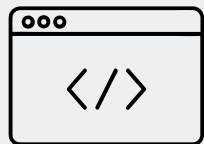
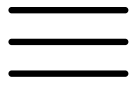
Escritura

Se almacenan los elementos en memoria con la función `write()`, se recibe la posición en memoria del valor a guardar y el tamaño en bytes del valor.

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    fstream archivo("archivo.bin", ios::out | ios::binary);
    int valor = 10;
    if(archivo) {
        archivo.write((char*) &valor, sizeof(int));
        archivo.close();
    }
    return 0;
}
```



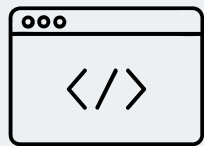
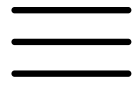
Lectura

Se define una variable como contenedor para la cantidad de bytes que se van a extraer desde el archivo. Se utiliza la función read()

```
#include <iostream>
#include <fstream>

using namespace std;

int main(){
    fstream archivo("archivo.bin", ios::in | ios::binary);
    int valor;
    if(archivo){
        archivo.read((char*) &valor, sizeof(int));
        cout << valor << endl; // 10
        archivo.close();
    }
    return 0;
}
```



Flags (modificadores)

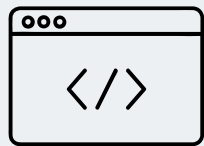
Constantes del compilador que cambian el comportamiento de las funciones del sistema

ios::in

ios::out

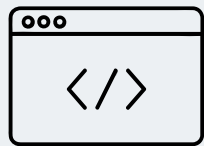
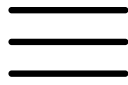
ios::app

ios::binary



Funciones de archivos binarios especializados para moverse dentro del stream del archivo para leer y escribir bytes en específico de este.

[illegible]

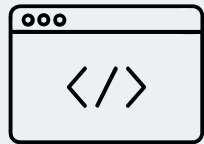
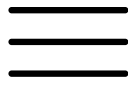


Seek

Navegar hasta una cantidad de bytes en específico sobre el stream.

- `ios::beg` (inicio del stream)
 - `ios::cur` (posición actual)
 - `ios::end` (final del stream)
-
- `seekg` para lectura
 - `seekp` para escritura

```
archivo.seekg(10, ios::beg);  
archivo.seekg(1, ios::cur);  
archivo.seekg(2, ios::end);
```

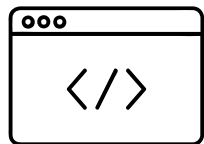
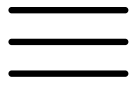


Tell

Obtener la posición actual del lugar señalado por el stream sobre el archivo

- `tellg()` para lectura
- `tellp()` para escritura

```
int pos = archivo.tellg();
```



Actividad:

Biblioteca: Crear un sistema en C++ que simule una base de datos de libros de una biblioteca de forma que se pueda alterar el stock de un libro al ser prestado y aumentarlo a solicitud.

Modificar el stock a traves del stream usando un archivo binario con funciones de acceso.





Gracias

Programación Avanzada